

Trie | (Display Content)

Trie is an efficient information retrieval data structure. In our [previous](#) post on trie we have discussed about basics of trie and how to insert and search a key in trie. In this post we will discuss about displaying all of the content of a trie. That is, to display all of the keys present in the Trie.

Examples:

```
Input: If Trie is      root
      /   \   \
      t    a    b
      |    |    |
      h    n    y
      |    |   \  |
      e    s   y  e
    /  |  |
    i  r  w
    |  |  |
    r  e  e
        |
        r
```

Output: Contents of Trie:

```
answer
any
bye
their
there
```

The idea to do this is to start traversing from the root node of trie, whenever we find a NON-NULL child node, we add parent key of child node in the “string str” at the current index(level) and then recursively call the same process for the child node and same goes on till we find the node which is a leafnode, which actually marks the end of the string.

Below is the C++ implementation of above idea:

```
// CPP program to display content of Trie
#include <iostream>
#include <string.h>
#define alpha_size 26
#define ARRAY_SIZE(a) sizeof(a) / sizeof(a[0])

using namespace std;

// Trie node
struct TrieNode
{
    struct TrieNode* children[alpha_size];

    bool isLeaf;
};

// Returns new trie node (initialized to NULLs)
struct TrieNode* createNode()
{
    struct TrieNode* pNode = new TrieNode;

    for (int i = 0; i < alpha_size; i++)
        pNode->children[i] = NULL;

    pNode->isLeaf = false;

    return pNode;
};

// function to insert a node in Trie
void insert_node(struct TrieNode* root, char* key)
{
    int level;
    int length = strlen(key);
    struct TrieNode* pCrawl = root;

    for (level = 0; level < length; level++)
    {
        int index = key[level] - 'a';

        if (pCrawl->children[index] == NULL)
            pCrawl->children[index] = createNode();

        pCrawl = pCrawl->children[index];
    }
}
```

```
    }

    pCrawl->isLeaf = true;
}

// function to check if current node is leaf node or not
bool isLeafNode(struct TrieNode* root)
{
    return root->isLeaf != false;
}

// function to display the content of Trie
void display(struct TrieNode* root, char str[], int level)
{
    // If node is leaf node, it indicates end
    // of string, so a null character is added
    // and string is displayed
    if (isLeafNode(root))
    {
        str[level] = '\0';
        cout << str << endl;
    }

    int i;
    for (i = 0; i < alpha_size; i++)
    {
        // if NON NULL child is found
        // add parent key to str and
        // call the display function recursively
        // for child node
        if (root->children[i])
        {
            str[level] = i + 'a';
            display(root->children[i], str, level + 1);
        }
    }
}
```

```
// Driver program to test above functions
int main()
{
    // Keys to be inserted in Trie
    char keys[][8] = { "the", "a", "there", "answer",
                      "any", "by", "bye", "their" };

    struct TrieNode* root = createNode();

    // Inserting keys in Trie
    for (int j = 0; j < ARRAY_SIZE(keys); j++)
        insert_node(root, keys[j]);

    int level = 0;
    char str[20];

    // Displaying content of Trie
    cout << "Content of Trie: " << endl;
    display(root, str, level);

}
```

Output:

Content of Trie:

a

answer

any

by

bye

the

their

there